3.9.4、凸性缺陷、点到轮廓、形状匹配

[OpenCV: Contours : More Functions]
https://docs.opencv.org/4.x/d5/d45/tutorial_py_contours_more_functions.html

这个教程主要教三个非常实用的高级功能：

1. 凸性缺陷 (Convexity Defects) —— 找出轮廓上“凹进去”的部分（比如手指之间的缝隙）
2. 点到多边形测试 (Point Polygon Test) —— 判断一个点在轮廓里面、外面，还是在边界上，并计算距离
3. 形状匹配 (Shape Matching) —— 比较两个轮廓的相似度（常用于识别相同或相似的形状）

```
import cv2 as cv
import numpy as np

img = cv.imread('your_image.jpg')          # 推荐用有凹陷的图片，比如手掌或星星
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
ret, thresh = cv.threshold(gray, 127, 255, 0)
contours, _ = cv.findContours(thresh, 2, 1)   # 注意这里用 cv.RETR_EXTERNAL 或 2
都行
cnt = contours[0]   # 取最大的轮廓（实际项目中建议按面积排序取最大）
```

1. 凸性缺陷（Convexity Defects）

概念：
即使我们有了凸包（Convex Hull），真实轮廓往往会有“凹陷”。这些从凸包“凹进去”的部分就叫凸性缺陷。典型应用：手指检测（指缝就是缺陷）、缺陷检测等。步骤：

- 先计算凸包，但必须设置 returnPoints=False（返回索引，而不是坐标）
- 再用 cv.convexityDefects() 计算缺陷

```
# 关键点：必须 returnPoints=False
hull = cv.convexHull(cnt, returnPoints=False)

# 计算凸性缺陷
defects = cv.convexityDefects(cnt, hull)

# defects 的形状是 (N, 1, 4)，每一行代表一个缺陷
# 四个值：[start_index, end_index, farthest_index, depth]
```

```
#tuple跟列表差不多
for i in range(defects.shape[0]):
    s, e, f, d = defects[i, 0]          # 解包四个值
    start = tuple(cnt[s][0])            # 缺陷起始点，坐标列表（x,y）
    end   = tuple(cnt[e][0])            # 缺陷结束点
    far   = tuple(cnt[f][0])            # 缺陷最深点（最凹的点）

    cv.line(img, start, end, [0, 255, 0], 2)      # 绿色线连接起点和终点
    cv.circle(img, far, 5, [0, 0, 255], -1)       # 红色点标记最深位置
```

思路：阈值化得到二值图像thresh----找轮廓contours----取一个轮廓cnt----计算凸包hull----计算凹陷defects

1.1. cnt 到底是什么?

cnt 是单个轮廓（Contour），它来自 cv.findContours() 返回的 contours 列表。具体来说：

- contours 是一个 Python list（列表），里面装了很多个轮廓。
- cnt = contours[0] → 取出列表中的第一个轮廓（通常我们取最大的那个）。
  cnt 的真实类型和形状：
- 类型：NumPy ndarray（NumPy 数组）
- 形状（shape）：通常是 (N, 1, 2)
  - N = 这个轮廓上有多少个边界点（顶点数量）
  - 1 = OpenCV 内部的维度（CHAIN_APPROX_SIMPLE 等方法导致的）
  - 2 = 每个点有 x 和 y 两个坐标
    怎么理解这个形状？ 想象一个轮廓由很多点连成一条封闭的曲线：
- 第 0 个点：cnt[0][0] → [x0, y0]
- 第 1 个点：cnt[1][0] → [x1, y1]
- ...
- 第 i 个点：cnt[i][0] → [xi, yi]

1.2. defects 的形状是 (N, 1, 4) 是什么意思?

defects = cv.convexityDefects(cnt, hull) 返回的就是凸性缺陷的信息。形状 (N, 1, 4) 的含义：

- N = 这张图里检测到的凸性缺陷数量（比如手掌通常有 4 个指缝 → N≈4）
- 1 = OpenCV 内部存储方式（类似 cnt 的 (N,1,2)）
- 4 = 每个缺陷用 4 个数字来描述
  这 4 个数字到底代表什么？（最重要！）对于每一个缺陷，defects[i, 0] 会给你一个长度为 4 的数组：
  python

```python
s, e, f, d = defects[i, 0]
```

- s（start_index）：缺陷段在凸包上的起始点索引（在 cnt 中的位置）
- e（end_index）：缺陷段在凸包上的结束点索引
- f（farthest_index）：轮廓上最凹进去的点（离凸包最远的点）的索引
- d（depth）：最凹点到凸包边界的近似距离（注意：实际像素距离要除以 256，即 d / 256.0）

2. 点到多边形测试（Point Polygon Test）

功能：计算一个点到轮廓的最短距离，同时告诉你点在内部、外部还是边界上。

```python
# measureDist=True  → 返回带符号的实际距离（推荐用于可视化）
dist = cv.pointPolygonTest(cnt, (50, 50), True)
print(dist)   # >0 在内部，<0 在外部，=0 在边界上
```

性能优化：

```python
# 如果你只想知道“里面还是外面”，用 False 会快 2~3 倍
inside = cv.pointPolygonTest(cnt, (50, 50), False)
# 返回 +1（内部）、-1（外部）、0（边界）
```

3. 形状匹配（Shape Matching）

功能：比较两个轮廓的相似程度，返回一个数值，越小越相似。基于 Hu 矩（Hu Moments），具有平移、旋转、缩放不变性，非常强大。

```python
ret = cv.matchShapes(cnt1, cnt2, 1, 0.0)
print("相似度距离:", ret)   # 越接近 0 越相似
```

- 相同形状：接近 0
- 非常相似：通常 < 0.01
- 不同形状：可能 > 0.1 甚至更大
  完整形状匹配示例（对比两张图片）：

```
img1 = cv.imread('star.jpg', 0)
img2 = cv.imread('star2.jpg', 0)

_, thresh1 = cv.threshold(img1, 127, 255, 0)
_, thresh2 = cv.threshold(img2, 127, 255, 0)

contours1, _ = cv.findContours(thresh1, 2, 1)
contours2, _ = cv.findContours(thresh2, 2, 1)

cnt1 = contours1[0]
cnt2 = contours2[0]

similarity = cv.matchShapes(cnt1, cnt2, 1, 0.0)
print(f"形状匹配距离: {similarity}")
```

方法参数常用 1（CV_CONTOURS_MATCH_I1），其他方法可以查文档。